

令，以及各个操作的时序特性。例如，编译器必须知道时序信息，才能够确定是用一条乘法指令，还是用移位和加法的某种组合。现代计算机用复杂的技术来处理机器级程序，并行地执行许多指令，执行顺序还可能不同于它们在程序中出现的顺序。程序员必须理解这些处理器是如何工作的，从而调整他们的程序以获得最大的速度。基于 Intel 和 AMD 处理器最近的设计，我们提出了这种机器的一个高级模型。我们还设计了一种图形数据流(data-flow)表示法，可以使处理器对指令的执行形象化，我们还可以利用它预测程序的性能。

了解了处理器的运作，我们就可以进行程序优化的第二步，利用处理器提供的指令级并行(instruction-level parallelism)能力，同时执行多条指令。我们会讲述几个对程序的变化，降低一个计算的不同部分之间的数据相关，增加并行度，这样就可以同时执行这些部分了。

我们以对优化大型程序的问题的讨论来结束这一章。我们描述了代码剖析程序(profiler)的使用，代码剖析程序是测量程序各个部分性能的工具。这种分析能够帮助找到代码中低效率的地方，并且确定程序中我们应该着重优化的部分。

在本章的描述中，我们使代码优化看起来像按照某种特殊顺序，对代码进行一系列转换的简单线性过程。实际上，这项工作远非这么简单。需要相当多的试错法试验。当我们进行到后面的优化阶段时，尤其是这样，到那时，看上去很小的变化会导致性能上很大的变化。相反，一些看上去很有希望的技术被证明是无效的。正如后面的例子中会看到的那样，要确切解释为什么某段代码序列具有特定的执行时间，是很困难的。性能可能依赖于处理器设计的许多细节特性，而对此我们所知甚少。这也是为什么要尝试各种技术的变形和组合的另一个原因。

研究程序的汇编代码表示是理解编译器以及产生的代码会如何运行的最有效手段之一。仔细研究内循环的代码是一个很好的开端，识别出降低性能的属性，例如过多的内存引用和对寄存器使用不当。从汇编代码开始，我们还可以预测什么操作会并行执行，以及它们会如何使用处理器资源。正如我们会看到的，常常通过确认关键路径(critical path)来决定执行一个循环所需要的时间(或者说，至少是一个时间下界)。所谓关键路径是在循环的反复执行过程中形成的数据相关链。然后，我们会回过头来修改源代码，试着控制编译器使之产生更有效率的实现。

大多数编译器，包括 GCC，一直都在更新和改进，特别是在优化能力方面。一个很有用的策略是只重写程序到编译器由此就能产生有效代码所需要的程度就好了。这样，能尽量避免损害代码的可读性、模块性和可移植性，就好像我们使用的是具有最低能力的编译器。同样，通过测量值和检查生成的汇编代码，反复修改源代码和分析它的性能是很有帮助的。

对于新手程序员来说，不断修改源代码，试图欺骗编译器产生有效的代码，看起来很奇怪，但这确实是编写很多高性能程序的方式。比较于另一种方法——用汇编语言写代码，这种间接的方法具有的优点是：虽然性能不一定是最好的，但得到的代码仍然能够在其他机器上运行。

5.1 优化编译器的能力和局限性

现代编译器运用复杂精细的算法来确定一个程序中计算的是什么值，以及它们是被如何使用的。然后会利用一些机会来简化表达式，在几个不同的地方使用同一个计算，以及降低一个给定的计算必须被执行的次数。大多数编译器，包括 GCC，向用户提供了一些对它们所使用的优化的控制。就像在第 3 章中讨论过的，最简单的控制就是指定优化级

别。例如，以命令行选项“-Og”调用GCC是让GCC使用一组基本的优化。以选项“-O1”或更高(如“-O2”或“-O3”)调用GCC会让它使用更大量的优化。这样做可以进一步提高程序的性能，但是也可能增加程序的规模，也可能使标准的调试工具更难对程序进行调试。我们的表述，虽然对于大多数使用GCC的软件项目来说，优化级别-O2已经成为了被接受的标准，但是还是主要考虑以优化级别-O1编译出的代码。我们特意限制了优化级别，以展示写C语言函数的不同方法如何影响编译器产生代码的效率。我们会发现可以写出的C代码，即使用-O1选项编译得到的性能，也比用可能的最高的优化等级编译一个更原始的版本得到的性能好。

编译器必须很小心地对程序只使用安全的优化，也就是说对于程序可能遇到的所有可能的情况，在C语言标准提供的保证之下，优化后得到的程序和未优化的版本有一样的行为。限制编译器只进行安全的优化，消除了造成不希望的运行时行为的一些可能的原因，但是这也意味着程序员必须花费更大的力气写出编译器能够将之转换成有效机器代码的程序。为了理解决定一种程序转换是否安全的难度，让我们来看看下面这两个过程：

```

1 void twiddle1(long *xp, long *yp)
2 {
3     *xp += *yp;
4     *xp += *yp;
5 }
6
7 void twiddle2(long *xp, long *yp)
8 {
9     *xp += 2* *yp;
10 }
```

乍一看，这两个过程似乎有相同的行为。它们都是将存储在由指针yp指示的位置处的值两次加到指针xp指示的位置处的值。另一方面，函数twiddle2效率更高一些。它只要求3次内存引用(读*xp，读*yp，写*xp)，而twiddle1需要6次(2次读*xp，2次读*yp，2次写*xp)。因此，如果要编译器编译过程twiddle1，我们会认为基于twiddle2执行的计算能产生更有效的代码。

不过，考虑xp等于yp的情况。此时，函数twiddle1会执行下面的计算：

```

3     *xp += *xp; /* Double value at xp */
4     *xp += *xp; /* Double value at xp */
```

结果是xp的值增加4倍。另一方面，函数twiddle2会执行下面的计算：

```

9     *xp += 2* *xp; /* Triple value at xp */
```

结果是xp的值增加3倍。编译器不知道twiddle1会如何被调用，因此它必须假设参数xp和yp可能会相等。因此，它不能产生twiddle2风格的代码作为twiddle1的优化版本。

这种两个指针可能指向同一个内存位置的情况称为内存别名使用(memory aliasing)。在只执行安全的优化中，编译器必须假设不同的指针可能会指向内存中同一个位置。再看一个例子，对于一个使用指针变量p和q的程序，考虑下面的代码序列：

```

x = 1000; y = 3000;
*q = y; /* 3000 */
*p = x; /* 1000 */
t1 = *q; /* 1000 or 3000 */
```

t1 的计算值依赖于指针 p 和 q 是否指向内存中同一个位置——如果不是, t1 就等于 3000, 但如果是, t1 就等于 1000。这造成了一个主要的妨碍优化的因素, 这也是可能严重限制编译器产生优化代码机会的程序的一个方面。如果编译器不能确定两个指针是否指向同一个位置, 就必须假设什么情况都有可能, 这就限制了可能的优化策略。

 **练习题 5.1** 下面的问题说明了内存别名使用可能会导致意想不到的程序行为的方式。考虑下面这个交换两个值的过程:

```
1  /* Swap value x at xp with value y at yp */
2  void swap(long *xp, long *yp)
3  {
4      *xp = *xp + *yp; /* x+y      */
5      *yp = *xp - *yp; /* x+y-y = x */
6      *xp = *xp - *yp; /* x+y-x = y */
7  }
```

如果调用这个过程时 xp 等于 yp, 会有什么样的效果?

第二个妨碍优化的因素是函数调用。作为一个示例, 考虑下面这两个过程:

```
1  long f();
2
3  long func1() {
4      return f() + f() + f() + f();
5  }
6
7  long func2() {
8      return 4*f();
9  }
```

最初看上去两个过程计算的都是相同的结果, 但是 func2 只调用 f 一次, 而 func1 调用 f 四次。以 func1 作为源代码时, 会很想产生 func2 风格的代码。

不过, 考虑下面 f 的代码:

```
1  long counter = 0;
2
3  long f() {
4      return counter++;
5  }
```

这个函数有个副作用——它修改了全局程序状态的一部分。改变调用它的次数会改变程序的行为。特别地, 假设开始时全局变量 counter 都设置为 0, 对 func1 的调用会返回 $0+1+2+3=6$, 而对 func2 的调用会返回 $4 \cdot 0=0$ 。

大多数编译器不会试图判断一个函数是否没有副作用, 如果没有, 就可能被优化成像 func2 中的样子。相反, 编译器会假设最糟的情况, 并保持所有的函数调用不变。

旁注 用内联函数替换优化函数调用

包含函数调用的代码可以用一个称为内联函数替换 (inline substitution, 或者简称“内联 (inlining)”) 的过程进行优化, 此时, 将函数调用替换为函数体。例如, 我们可以通过替换掉对函数 f 的四次调用, 展开 func1 的代码:

```
1  /* Result of inlining f in func1 */
2  long func1in() {
3      long t = counter++; /* +0 */
```



```

4      t += counter++;      /* +1 */
5      t += counter++;      /* +2 */
6      t += counter++;      /* +3 */
7      return t;
8  }
```

这样的转换既减少了函数调用的开销，也允许对展开的代码做进一步优化。例如，编译器可以统一 `func1` 中对全局变量 `counter` 的更新，产生这个函数的一个优化版本：

```

1  /* Optimization of inlined code */
2  long funclopt() {
3      long t = 4 * counter + 6;
4      counter += 4;
5      return t;
6  }
```

对于这个特定的函数 `f` 的定义，上述代码忠实地重现了 `func1` 的行为。

GCC 的最近版本会尝试进行这种形式的优化，要么是被用命令行选项 “`-finline`” 指示时，要么是使用优化等级 `-O1` 或者更高的等级时。遗憾的是，GCC 只尝试在单个文件中定义的函数的内联。这就意味着它将无法应用于常见的情况，即一组库函数在一个文件中被定义，却被其他文件内的函数所调用。

在某些情况下，最好能阻止编译器执行内联替换。一种情况是用符号调试器来评估代码，比如 GDB，如 3.10.2 节描述的一样。如果一个函数调用已经用内联替换优化过了，那么任何对这个调用进行追踪或设置断点的尝试都会失败。还有一种情况是用代码剖析的方式来评估程序性能，如 5.14.1 节讨论的一样。用内联替换消除的函数调用是无法被正确剖析的。

在各种编译器中，就优化能力来说，GCC 被认为是胜任的，但是并不是特别突出。它完成基本的优化，但是它不会对程序进行更加“有进取心的”编译器所做的那种激进变换。因此，使用 GCC 的程序员必须花费更多的精力，以一种简化编译器生成高效代码的任务的方式来编写程序。

5.2 表示程序性能

我们引入度量标准每元素的周期数 (Cycles Per Element, CPE)，作为一种表示程序性能并指导我们改进代码的方法。CPE 这种度量标准帮助我们在更细节的级别上理解迭代程序的循环性能。这样的度量标准对执行重复计算的程序来说是很适当的，例如处理图像中的像素，或是计算矩阵乘积中的元素。

处理器活动的顺序是由时钟控制的，时钟提供了某个频率的规律信号，通常用千兆赫兹 (GHz)，即十亿周期每秒来表示。例如，当表明一个系统有 “4GHz” 处理器，这表示处理器时钟运行频率为每秒 4×10^9 个周期。每个时钟周期的时间是时钟频率的倒数。通常是以纳秒 (nanosecond, 1 纳秒等于 10^{-9} 秒) 或皮秒 (picosecond, 1 皮秒等于 10^{-12} 秒) 为单位的。例如，一个 4GHz 的时钟其周期为 0.25 纳秒，或者 250 皮秒。从程序员的角度来看，用时钟周期来表示度量标准要比用纳秒或皮秒来表示有帮助得多。用时钟周期来表示，度量值表示的是执行了多少条指令，而不是时钟运行得有多快。

许多过程含有在一组元素上迭代的循环。例如，图 5-1 中的函数 `psum1` 和 `psum2` 计算的都是一个长度为 n 的向量的前置和 (prefix sum)。对于向量 $\vec{a} = \langle a_0, a_1, \dots, a_{n-1} \rangle$ ，前置和 $\vec{p} = \langle p_0, p_1, \dots, p_{n-1} \rangle$ 定义为